Lab Sheet 1: DDL Commands – CREATE, ALTER and DROP

Perform the following SQL tasks using suitable examples:

1. Create a database named CollegeDB.
2. Create a Student table with columns: Student_ID, Name, Age, Course, and City.
3. Create an Employee table with Emp_ID, Emp_Name, Department, and Salary.
4. Create a Department table with Dept_ID, Dept_Name, and Location.
5. Create a Product table with Product_ID, Product_Name, Price, and Quantity.
6. Create a Customer table with Customer_ID, Customer_Name, Email, and Phone.
7. Create a Course table with Course_ID, Course_Name, Duration, and Fees.
8. Create a Faculty table with Faculty_ID, Faculty_Name, Subject, and Salary.
9. Create a Library table with Book_ID, Book_Name, Author, and Price.
10. Create a Department table with Dept_ID as the **Primary Key**.
11. Create a Student table with Student_ID as the **Primary Key** and Name as NOT NULL.
12. Create an Employee table with a UNIQUE constraint on Email.
13. Create a Product table with a CHECK constraint that Price > 0.
14. Add a new column Email to the Student table using ALTER TABLE.
15. Add a Phone column to the Student table.
16. Modify the datatype/size of the Name column in the Student table.
17. Rename the Student table to Student_Details.
18. Rename the column City to Address.
19. Add a Salary column to the Employee table.
20. Drop the Phone column from the Student table.
21. Drop the Email column from the Student table.
22. Remove all records from a table using TRUNCATE.
23. Drop the Course table from the database.
24. Drop the Customer table from the database.
25. Create a complete Student table using **Primary Key, NOT NULL, UNIQUE and CHECK constraints**, and demonstrate suitable ALTER and DROP operations.

Lab Sheet 2: DML Commands – INSERT, UPDATE and DELETE

Perform the following SQL tasks using suitable examples:

1. Insert one record into the Student table.
2. Insert five student records using separate INSERT statements.
3. Insert multiple records into the Student table using a single statement.
4. Insert records into the Employee table.
5. Insert records into the Department table.
6. Insert records into the Product table.
7. Insert records into the Customer table.
8. Insert records into the Course table.
9. Insert a student record with values for all columns.
10. Insert a student record by specifying only selected columns.
11. Update the city of a particular student.
12. Update the salary of a particular employee.
13. Increase the salary of all employees by 10%.
14. Increase the price of all products by 5%.
15. Change the course of a particular student.
16. Update the department of an employee.
17. Update the city of all students from Meerut to Delhi.
18. Update the fees of a particular course.
19. Update multiple columns of a student simultaneously.
20. Delete a particular student record using Student_ID.
21. Delete all students belonging to a particular city.
22. Delete employees whose salary is below a specified amount.
23. Delete products whose quantity is zero.
24. Delete records satisfying multiple conditions using AND/OR.
25. Perform a complete sequence of **INSERT, UPDATE and DELETE** operations on a Student table and display the table after each operation.

Lab Sheet 3: SQL Functions

A. Number Functions

1. Demonstrate ABS() with suitable examples.

2. Demonstrate ROUND() with suitable examples.
3. Demonstrate CEIL() and FLOOR().
4. Demonstrate MOD() with suitable examples.
5. Calculate the rounded salary of employees.

B. Aggregate Functions

6. Find the total number of students using COUNT().
7. Find the total salary of all employees using SUM().
8. Find the average salary using AVG().
9. Find the highest salary using MAX().
10. Find the lowest salary using MIN().

C. Character Functions

11. Demonstrate UPPER() on student names.
12. Demonstrate LOWER() on student names.
13. Demonstrate LENGTH() on student names.
14. Demonstrate CONCAT() using first and last names.
15. Demonstrate SUBSTRING()/SUBSTR() with suitable examples.

D. Conversion Functions

16. Convert a number into a character/string value.
17. Convert a string into a numeric value.
18. Convert a date value into a different format.
19. Demonstrate CAST() with suitable examples.
20. Demonstrate CONVERT() where supported by the DBMS.

E. Date Functions

21. Display the current date using a suitable date function.
22. Extract the year, month and day from a date.
23. Find the difference between two dates.
24. Add a specified number of days to a date.
25. Display employees' joining dates in a required date format.

Note: Function names can vary slightly between MySQL, Oracle and SQL Server. Since you use **MySQL Workbench**, these can be framed specifically for MySQL.

Lab Sheet 4: SQL Operators

Arithmetic Operators

1. Perform addition of two numbers using +.
2. Perform subtraction using -.
3. Perform multiplication using *.
4. Perform division using /.
5. Calculate the total amount as Price × Quantity.

Logical Operators

6. Demonstrate AND with two conditions.
7. Demonstrate OR with two conditions.
8. Demonstrate NOT with a condition.
9. Find students belonging to a specific course **AND** city.
10. Find employees belonging to either of two departments.

Comparison Operators

11. Demonstrate the = operator.
12. Demonstrate the > operator.
13. Demonstrate the < operator.
14. Demonstrate >= and <=.
15. Demonstrate the <>/!= operator.

Special Operators

16. Demonstrate BETWEEN for salary range.
17. Demonstrate IN for selecting multiple cities.
18. Demonstrate NOT IN.
19. Demonstrate LIKE for names beginning with a particular letter.
20. Demonstrate IS NULL and IS NOT NULL.

Set Operations

21. Demonstrate UNION using two compatible queries.
22. Demonstrate UNION ALL.
23. Demonstrate INTERSECT where supported by the DBMS.
24. Demonstrate EXCEPT/MINUS where supported.
25. Perform a suitable problem using multiple operators together, such as **Arithmetic + Logical + Comparison + Special operators**.

Lab Sheet 5: SQL Joins

Use suitable tables such as Student, Department, Course, and Employee to perform the following:

1. Demonstrate an **INNER JOIN** between Student and Department.
2. Display student names along with their department names using INNER JOIN.
3. Display employee names along with their department names.
4. Display students along with their course details.
5. Find students belonging to a particular department using INNER JOIN.
6. Demonstrate a **LEFT OUTER JOIN** between Student and Department.
7. Display all students, including those who do not have a matching department.
8. Demonstrate a **RIGHT OUTER JOIN** between Student and Department.
9. Display all departments, including departments having no students.
10. Find students who are not assigned to any department using LEFT JOIN.
11. Find departments having no students using RIGHT/LEFT JOIN.
12. Demonstrate a **FULL OUTER JOIN** concept using suitable queries/workaround where supported.
13. Demonstrate a **CROSS JOIN** between Student and Course.
14. Calculate the number of possible Student-Course combinations using CROSS JOIN.
15. Demonstrate a **NATURAL JOIN** between two suitable tables.
16. Compare the result of NATURAL JOIN and INNER JOIN.
17. Perform a **SELF JOIN** on an Employee table to show employee-manager relationships.
18. Display employees along with their managers using SELF JOIN.
19. Join three tables—Student, Department and Course.
20. Display student name, course name and department name using multiple joins.
21. Find employees whose salary is greater than the average salary of their department using joins/subqueries.
22. Find the department having the maximum number of students using joins and aggregation.
23. Display departments and the total number of students in each department.
24. Display courses along with the number of students enrolled in each course.
25. Create a complete query using **INNER JOIN, LEFT JOIN, GROUP BY and aggregate functions** to generate a student-department-course report.